

4주차 Kafka 전달 보장과 중복 방지

4주차 · Kafka

- ☐ 3주차에 띄운 Kafka 브로커로 민원 이벤트를 보내고, 전달 실패가 저장소의 중복으로 이어지는 과정을 확인함
 - ☐ practice/chapter4에서 파티션·커밋·유실·재수신을 관찰함
 - ☐ practice/chapter4에서 재수신된 이벤트를 유니크 제약과 UPSERT로 흡수함
- ☐ 상세 이론은 lecture/docs/chapter4.md와 lecture/docs/chapter5.md에 그대로 유지함

1. 오늘의 질문

- ☐ Kafka 토픽에 넣은 순서가 전체에서 그대로 보존되는가
- ☐ 커밋 시점이 왜 유실과 중복을 서로 다른 방향으로 만드는가
- ☐ 같은 이벤트를 판정하는 키는 언제 만들어야 하는가
- ☐ 여러 번 전달된 이벤트를 한 번 반영된 상태로 만드는 방법은 무엇인가

2. 파티션이 순서 보장 범위를 정한다

- ☐ Kafka 토픽(topic)은 하나 이상의 파티션(partition)으로 나뉨
 - ☐ 파티션은 이벤트가 뒤에만 추가되는 로그임
 - ☐ 오프셋(offset)은 파티션 안에서만 증가함
 - ☐ 따라서 순서도 토픽 전체가 아니라 파티션 안에서만 보장됨
- ☐ 생산자(Producer)가 키를 지정하면 같은 키의 이벤트가 같은 파티션으로 감
 - ☐ 지역구를 키로 쓰면 같은 지역구의 접수 순서를 보존함
 - ☐ 민원 ID를 키로 쓰면 한 민원의 접수·배정·처리 순서를 보존함
 - ☐ 키 선택은 부하 분산 설정이 아니라 무엇끼리 순서를 지킬지 정하는 업무 판단임

표 4.1 실제 이벤트 네 건의 파티션과 오프셋

이벤트	키	파티션	오프셋
#1 불법주차	중구	1	0
#2 도로파손	강남구	1	1
#3 도로파손	성동구	0	0
#4 가로등고장	중구	1	2

- ☐ 표 4.1에서 전체 세 번째 이벤트인 #3의 오프셋은 0임
 - ☐ 파티션 0에서는 첫 번째 이벤트이기 때문임
 - ☐ 중구 두 건은 파티션 1에 함께 놓여 그 안에서 순서가 보존됨

- 실습은 8개 지역구를 키로 200건을 3개 파티션에 보냄
 - 실측 분포는 61·76·63건이고 균등값은 66.7건임
 - 키가 2:3:3개로 나뉘어 파티션별 부하가 달라짐
 - `key_to_single_partition: true`는 같은 지역구가 한 파티션에만 들어갔다는 증거임
- 파티션 수를 3개에서 4개로 바꾸면 해시값을 나누는 수가 달라짐
 - 같은 지역구가 다른 파티션으로 이동할 수 있음
 - 감사에 필요한 지역별 순서가 증설 시점에서 끊길 수 있으므로 초기 설계에서 변경 영향을 기록함

3. 커밋 시점이 유실과 중복을 가른다

- 소비자(Consumer)는 처리 위치를 커밋된 오프셋으로 저장함
 - 재시작한 소비자는 마지막 커밋 위치부터 다시 읽음
- 커밋을 처리보다 먼저 하면 최대 한 번(at-most-once) 전달이 됨
 - 처리 중 장애가 나면 이미 커밋한 이벤트를 다시 읽지 못함
 - 중복은 없지만 유실이 발생할 수 있음
- 처리를 커밋보다 먼저 하면 최소 한 번(at-least-once) 전달이 됨
 - 처리 뒤 커밋 전에 장애가 나면 같은 이벤트를 다시 읽음
 - 유실은 막지만 중복이 발생할 수 있음

표 4.2 커밋 순서에 따른 실측 결과

시나리오	생산	처리 기록	고유 이벤트	중복	유실
A. 정상 경로	200	200	200	0	0
B. 커밋 후 처리	30	10	10	0	20
C. 처리 후 커밋	30	40	30	10	0

- B에서는 30건을 커밋한 뒤 10건만 처리하고 종료함
 - 재시작 뒤 나머지 20건을 다시 읽지 못해 유실됨
 - 오류 로그가 없어도 생산 30건과 고유 처리 10건을 대사하면 유실을 찾을 수 있음
- C에서는 10건을 처리한 뒤 커밋 전에 종료함
 - 재시작 뒤 30건을 처음부터 읽어 처리 기록이 40건이 됨
 - 고유 이벤트는 30건이므로 재수신 중복은 10건임
- 공공 데이터는 교정 가능한 중복보다 복구할 수 없는 유실을 우선 피함
 - 기본 경로를 at-least-once로 두고 하류 저장소에서 중복을 제거함
 - 자동 커밋은 저장 완료 전에 오프셋을 이동시킬 수 있으므로 실습에서는 끄

4. 중복 제거 키와 저장소 제약이 반영을 지킨다

- 중복 제거 키(deduplication key)는 같은 이벤트의 반복 전달을 판정하는 기준임

- 생성 시점의 고유 ID는 재전송과 재수신을 같은 이벤트로 묶음
 - 전송 함수 안에서 매번 새 ID를 만들면 반복 전달이 서로 다른 이벤트로 보임
 - 기술적 반복 전달과 시민이 실제로 다시 접수한 별도 민원은 구분해야 함
- 애플리케이션에서 조회한 뒤 저장하는 방식은 경쟁 조건에 취약함
- 소비자 두 대가 동시에 조회하면 둘 다 값이 없다고 판단할 수 있음
 - 저장소의 유니크 제약은 한 위치에서 원자적으로 두 번째 삽입을 막음
- 충돌 처리 방식은 저장 목적에 따라 달라짐
- 실패는 중복 도착 자체가 오류일 때 사용함
 - 무시는 같은 내용을 버리지만 중복이 있었다는 기록도 남기지 않음
 - UPSERT는 한 행을 유지하면서 `seen_count`와 마지막 관찰 시각을 갱신함

```
INSERT INTO complaints upsert VALUES (?, ?, ?, ?, 1)
ON CONFLICT(event id) DO UPDATE SET
  last_seen_at = excluded.last_seen_at, seen_count = seen_count + 1
```

- 여러 번 전달되어도 업무 상태가 한 번 반영된 것과 같으면 멍등(idempotent)함
- Kafka의 `exactly-once`를 검토할 때 전달 보장과 반영 보장을 구분함
 - 네트워크 응답이 없으면 기록 실패와 응답 유실을 구분할 수 없으므로 정확히 한 번 전달만으로 문제를 해결할 수 없음
 - 저장소 제약과 멍등 쓰기를 결합해 정확히 한 번 반영된 결과를 만들

표 4.3 중복 스트림 40건의 저장 전략별 결과

단계	행 수	관찰 지표	해석
A. 제약 없는 삽입	40	중복 행 10	상류 중복이 그대로 저장됨
B. 유니크 제약과 무시	30	무시된 중복 10	업무 상태는 지키지만 발생 이력은 사라짐
C. UPSERT	30	총 관찰 40, 재관찰 이벤트 10	상태와 전달 이력을 함께 보존함
D. 전체 리플레이	30	총 관찰 80	행 수와 업무 상태가 재처리에 불변임

- C와 D의 업무 상태 지문은 `eed1522a7ca844bd`로 같음
- 행 수 30과 업무 필드가 리플레이 뒤에도 변하지 않음
 - 전달 이력인 총 관찰 수는 40에서 80으로 늘어나는 것이 정상임
 - 업무 상태와 전달 이력을 분리해야 멍등성을 정확히 검증할 수 있음

5. 실습 4주차 — 전달 실패부터 중복 제거까지

시작 전 준비

☐ Docker Desktop이 실행 중이어야 함

- 다음 명령이 Docker-포트 가상환경을 점검하고 Kafka와 Zookeeper를 준비함

```
python scripts/setup_practice.py 4
```

1단계 — Kafka 전달 실험

```
cd practice/chapter3 && docker compose up -d zookeeper kafka
cd ../chapter4
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter4.py
```

```
if mode == "at-most-once":
    consumer.commit()
process(batch)
if mode == "at-least-once":
    consumer.commit()
```

_전체 코드는 practice/chapter4/code/4-2-kafka-consumer.py, 오케스트레이션은 practice/chapter4/run_chapter4.py 참고_

☐ practice/chapter4/data/output/ch4_delivery_report.json에서 다음을 확인함

- 파티션 분포가 61-76-63인지 확인함
- B의 유실 20건과 C의 중복 10건을 커밋 순서와 연결함
- 실행 환경에 따라 건수나 지연이 달라지면 본인 산출물의 값을 사용함

2단계 — 저장소 중복 제거 실험

```
cd ../chapter5
python3 run_dedup.py
```

```
cur = conn.execute("INSERT OR IGNORE INTO complaints unique VALUES (?, ?, ?, ?)", row)
ignored += 1 - cur.rowcount
```

전체 코드는 practice/chapter4/code/4-3-deduplicate-events.py 참고

☐ practice/chapter4/data/output/ch4_dedup_report.json에서 다음을 확인함

- A는 40행이고 C는 30행인지 확인함
- C와 D의 업무 상태 지문이 같은지 확인함
- 총 관찰 수가 40에서 80으로 바뀌어도 업무 상태가 불변인 이유를 설명함

6. 핵심 요약

- ☐ 순서는 파티션 안에서만 보존되며 키가 그 범위를 정함
- ☐ 커밋을 먼저 하면 유실, 처리를 먼저 하면 재수신 중복이 발생할 수 있음
- ☐ 중복 제거 키는 생성 시점에 만들고 재시도 경로에서 바꾸지 않음
- ☐ 유니크 제약과 UPSERT로 at-least-once 전달을 멍등하게 반영함

7. 과제

- ☐ 두 실습을 실행해 다음 파일을 한 번의 LMS 제출에 첨부함
 - `practice/chapter4/data/output/ch4_delivery_report.json`
 - `practice/chapter4/data/output/ch4_dedup_report.json`
- ☐ 자기 산출물의 값을 사용해 다음을 답함
 1. B의 유실과 C의 중복을 각각 적고 커밋 시점 차이로 설명한다.
 2. 제약 없는 삽입과 UPSERT의 행 수 차이가 어느 재수신 중복과 대응하는지 설명한다.
 3. C와 D의 업무 상태 지문은 같고 총 관찰 수는 다른 이유를 업무 상태와 전달 이력으로 구분해 설명한다.
- ☐ 수업일 포함 7일 이내에 제출함

8. 더 알아보기

- ☐ Kafka의 상세 설명은 `lecture/docs/chapter4.md`에서 확인함
 - 토픽 명명·파티션 키·보존 기간: §4.2
 - Producer·Consumer와 커밋: §4.3·§4.4
 - 실패·재시도와 트랜잭션: §4.5
- ☐ 중복 방지의 상세 설명은 `lecture/docs/chapter5.md`에서 확인함
 - 중복 제거 키와 유니크 제약: §5.2·§5.3
 - watermark와 지연 이벤트: §5.4
 - Kafka 트랜잭션과 dedup 캐시: 보충 절